

Git Hands-On

Lab 1: Git Setup, Notepad++ Integration, and Adding a File to the Repository

Step 1: Setup Git Configuration

1. Check if Git client is installed properly

```
git --version    # displays installed Git version
```

2. Configure user level user ID and email ID

```
git config --global user.name "Your Name"    # sets global username
```

```
git config --global user.email "you@example.com"    # sets global email
```

3. Verify the configuration

```
git config --global --list    # lists all global configuration values
```

Step 2: Integrate Notepad++ as the Default Editor

1. Check if notepad++.exe runs from Git Bash

```
notepad++    # fails if notepad++ path is not set in environment variable
```

If Git Bash cannot recognize the command, add the Notepad++ installation path to the Windows PATH user variable via Control Panel -> System -> Advanced System Settings -> Environment Variables.

2. Reopen Git Bash and test again

```
notepad++    # opens Notepad++ from Git Bash after PATH is updated
```

3. Create an alias command for notepad++.exe

```
alias notepad++='winpty notepad++.exe'    # creates a short alias in the bash profile
```

4. Configure Notepad++ as the default editor

```
git config --global core.editor "notepad++ -multiinst -notabbar -nosession -noPlugin"    # sets Notepad++ as default editor
```

5. Verify Notepad++ is the default editor

```
git config --global -e    # -e opens the configured editor to confirm
```

Step 3: Add a File to the Source Code Repository

1. Create a new project folder and initialize repository

```
mkdir GitDemo && cd GitDemo    # creates and enters the project folder
```

```
git init    # initializes a new Git repository
```

2. Verify repository initialization

`ls -la` # lists hidden .git folder confirming the working directory is a Git repo

3. Create welcome.txt and add content

`echo "Welcome to Git" > welcome.txt` # creates file with content

4. Verify the file is created

`ls` # lists files in the working directory

5. Verify the content

`cat welcome.txt` # displays file content

6. Check the status

`git status` # shows welcome.txt as untracked

7. Track the file with Git

`git add welcome.txt` # stages the file for commit

8. Commit with a multi-line message using the default editor

`git commit` # opens Notepad++ to write the commit message

9. Check status after commit

`git status` # confirms there are no pending changes and the working directory is clean

10. Create a remote repository

Sign up on GitLab and create a remote repository named "GitDemo".

11. Connect the local repository to the remote repository

`git remote add origin https://gitlab.com/username/GitDemo.git` # connects the local Git repository to the remote GitLab repository named "origin"

12. Verify the remote repository connection

`git remote -v` # displays the configured remote repository URLs for fetch and push

13. Pull from the remote repository

`git pull origin master` # syncs local repository with remote

14. Push local changes to the remote repository

`git push -u origin master` # uploads local commits to the remote repository

Note:

The -u (or --set-upstream) option sets the upstream tracking branch. After this, you can simply use: git push, git pull, without specifying origin master every time.

Lab 2: Git Ignore

1. Create a .log file in the working directory

```
touch error.log # creates a sample log file
```

2. Create a log folder in the working directory

```
mkdir log # creates a sample log folder
```

3. Create and update the .gitignore file

```
touch .gitignore # creates the gitignore file
```

```
echo '*.log' >> .gitignore # ignores all files with .log extension
```

```
echo 'log/' >> .gitignore # ignores the entire log folder
```

4. Verify with git status

```
git status # confirms error.log and log/ no longer appear as untracked
```

5. Add and commit the .gitignore file

```
git add .gitignore # stages the gitignore file
```

```
git commit -m "Add gitignore file" # commits gitignore to local repository
```

6. Confirm working directory, local repository and Git status match

```
git status # shows clean working directory with ignored files excluded
```

Lab 3: Branching and Merging

Branching

1. Create a new branch

```
git branch GitNewBranch # creates a new branch named GitNewBranch
```

2. List all local and remote branches

```
git branch -a # lists all branches, * marks the current branch
```

3. Switch to the new branch and add files

```
git checkout GitNewBranch # switches to the new branch
```

```
echo "Branch content" > branchfile.txt # creates a file with content in the branch
```

4. Commit the changes to the branch

```
git add branchfile.txt # stages the new file
```

```
git commit -m "Add branchfile in GitNewBranch" # commits changes to the branch
```

5. Check the status

```
git status # confirms branch working directory is clean
```

Merging

1. Switch to master

```
git checkout master    # switches back to the master branch
```

2. List command line differences between trunk and branch

```
git diff master GitNewBranch    # shows differences between the two branches
```

3. View visual differences using P4Merge

```
git difftool -t p4merge master GitNewBranch    # opens P4Merge to visually compare branches
```

4. Merge the source branch into the trunk

```
git merge GitNewBranch    # merges GitNewBranch into master
```

5. Observe the log after merging

```
git log --oneline --graph --decorate    # shows commit history graph after merge
```

6. Delete the branch after merging

```
git branch -d GitNewBranch    # deletes the merged branch
```

```
git status    # confirms the branch no longer exists and master is clean
```

Lab 4: Conflict Resolution

1. Verify master is in a clean state

```
git status    # confirms master has no pending changes
```

2. Create branch GitWork and add hello.xml

```
git checkout -b GitWork    # creates and switches to the GitWork branch
```

```
echo "<hello>Branch Version</hello>" > hello.xml    # creates hello.xml in the branch
```

(git switch is the newer alternative to git checkout)

3. Update the content of hello.xml and observe status

```
echo "<hello>Updated Branch Version</hello>" > hello.xml    # updates file content
```

```
git status    # shows hello.xml as modified/untracked in the branch
```

4. Commit the changes to the branch

```
git add hello.xml    # stages hello.xml
```

```
git commit -m "Add hello.xml in GitWork branch"    # commits changes in the branch
```

5. Switch to master

```
git checkout master    # switches back to master
```

6. Add hello.xml to master with different content

`echo "<hello>Master Version</hello>" > hello.xml` # creates hello.xml in master with different content

7. Commit the changes to master

`git add hello.xml` # stages hello.xml in master

`git commit -m "Add hello.xml in master"` # commits changes in master

8. Observe the log for both branches

`git log --oneline --graph --decorate --all` # shows diverged history of master and GitWork

9. Check differences with git diff

`git diff master GitWork` # shows differences in hello.xml between branches

10. Visualize differences using P4Merge

`git difftool -t p4merge master GitWork` # opens P4Merge to compare hello.xml visually

11. Merge the branch into master

`git merge GitWork` # attempts to merge GitWork into master and raises a conflict

12. Observe the conflict markup

`cat hello.xml` # shows <<<<<<, =====, >>>>>> conflict markers in the file

13. Use the 3-way merge tool to resolve the conflict

`git mergetool` # opens P4Merge to resolve the conflict using a 3-way merge

14. Commit the resolved changes to master

`git add hello.xml` # stages the resolved file

`git commit -m "Resolve conflict in hello.xml"` # commits the merge resolution

15. Observe status and add backup file to .gitignore

`git status` # shows an orig backup file created by the merge tool as untracked

`echo '*.orig' >> .gitignore` # ignores merge tool backup files

16. Commit the changes to .gitignore

`git add .gitignore` # stages the updated gitignore file

`git commit -m "Update gitignore to exclude backup files"` # commits gitignore update

17. List all available branches

`git branch -a` # lists all local and remote branches

18. Delete the merged branch

`git branch -d GitWork` # deletes GitWork after it is merged into master

19. Observe the log

```
git log --oneline --graph --decorate # shows final commit history after  
conflict resolution
```

Lab 5: Clean Up and Push Back to Remote Git

1. Verify master is in a clean state

```
git status # confirms no pending changes in master
```

2. List all available branches

```
git branch -a # lists all local and remote branches
```

3. Pull the remote repository to master

```
git pull origin master # updates local master with the latest remote changes
```

4. Push pending changes to the remote repository

```
git push origin master # pushes local commits from previous lab to the remote  
repository
```

5. Verify changes are reflected in the remote repository

Log in to the GitLab web interface, open the "GitDemo" project, and confirm the pushed commits and files appear in the remote repository.